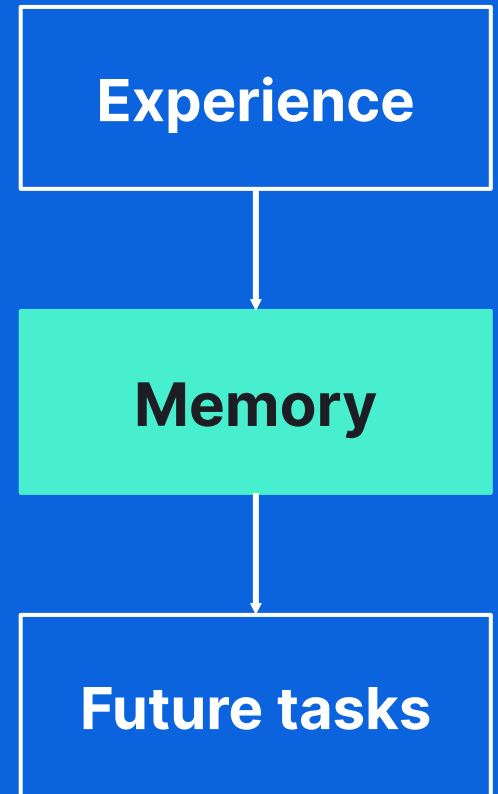


Building long-term agent memory in Elastic

Susan Shu Chang

Principal Data Scientist, Elastic

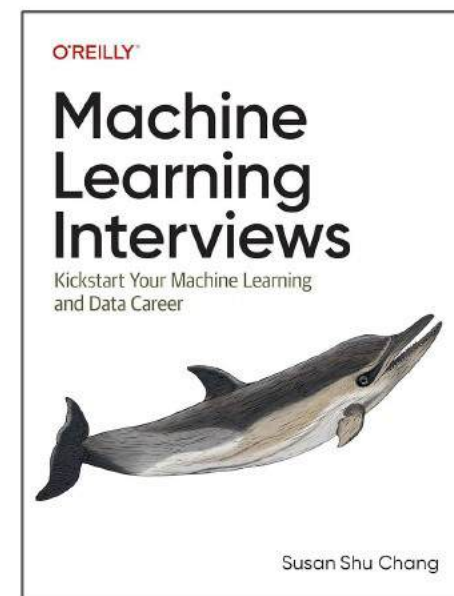


Hi!



Susan Shu Chang

- Principal Data Scientist at Elastic
- Spoke at 5+ PyCons around the world
- Author, Machine Learning Interviews (O'Reilly)



Web & app search / vector search / log search...

 Blog

○ Everything

↗ Productivity

 Career Advice

 AI/ML

 Open Source

 Business Hub

 Company

 JULY 31, 2023

Ask like a human: Implementing semantic search on Stack Overflow

Sem
of a

Engineering

Overview Backend Culture Data / ML Mobile

Engineering

Engineering Uber Predictions in Real Time with ELK

Using Elasticsearch queries, we can quickly see every action the user has done - this is a great way to see whether an account has been stolen, hijacked, or whether the user has done something naughty.

– Tim Pease, Operations Engineer, GitHub

Engineering

Tinder's migration to Elasticsearch 8

Posted on: April 1, 2025

Share this post



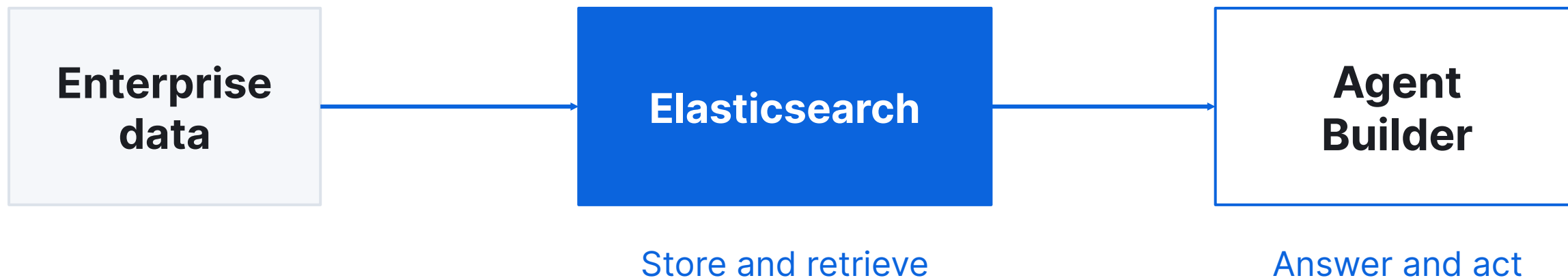
Today's topics

- 01** What is agent memory?
- 02** Storing and retrieving memory
- 03** Keeping memory up to date
- 04** Evaluating memory and what's next

What is agent memory?

Elastic connects agents to data and tools

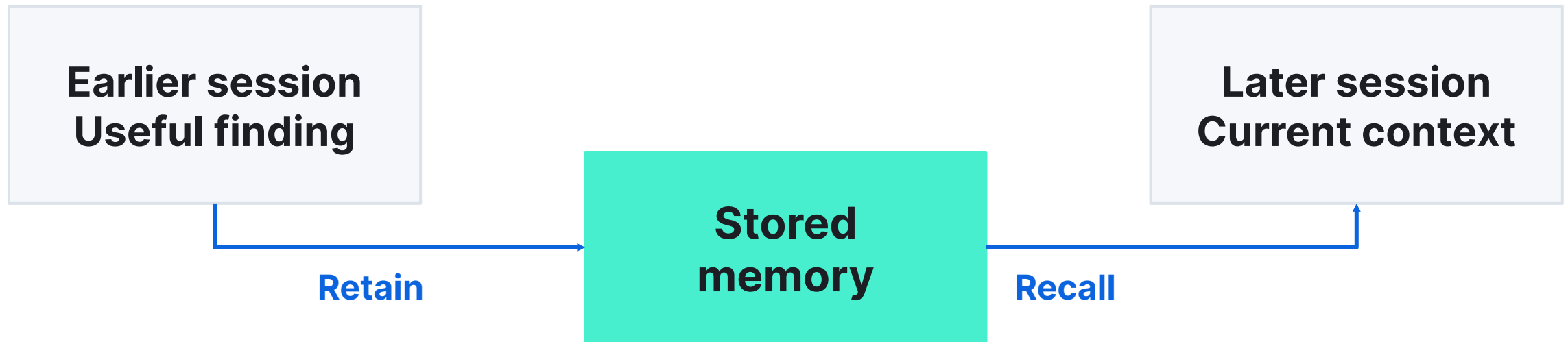
Search, observability, and security share a need for relevant information.



The same foundation can store and retrieve memories for later tasks.

Agents can use what they learned in earlier tasks

Long-term memory lives outside a single task's working context.



Memory becomes useful when it is selected and applied to the next task.

A correction should help the next task

Illustrative checkout incident

A correction

Retained finding

Useful later behavior



Does the correction still apply, and can this agent use it?

Different memories support different tasks

Conceptual categories, rather than a list of shipped product features

Type	What it captures	Example
Episodic	What happened	The incident ended after a rollback.
Semantic	A fact or preference	Payments owns checkout.
Procedural	How to approach a task	Check recent deployments first.

Storing and retrieving memory

Two records for the memory model

Current implementation direction: task knowledge written through remember

During the work

Session fact
A focused learning

Session synthesis

Session summary
What worked and why

Memory-enabled AI index

Existing index or data stream. Identity and provenance are recorded server-side.

Learn from experience. Recall it in a new chat.

Shortened dialogue showing the expected demo behavior

Chat 1 · Learn from experience and store in memory

USER

“For us, platinum tickets breach the SLO after 45 minutes, at any priority. Use this rule going forward. Which product has the most breaches?”

AGENT

“Search has 3 breaches. I’ve saved your definition for future questions.”

Chat 2 · Start a fresh chat

USER

“Which product currently has the most platinum SLO breaches?”

AGENT

“Using your saved definition, Search has 3 breaches in the current data.”

Memory can also answer “What does that mean?”

A fresh chat can ask for the saved definition directly

New conversation

USER

“What is the definition of a platinum SLO?”

AGENT

“Your saved rule says a platinum ticket breaches the SLO if it takes more than 45 minutes to resolve, regardless of priority.”

Recall the definition from memory. No ticket-count query needed.

Saving a new policy

Agent Builder demo



TU Test User · 2:56 PM

Our P0 escalation policy: any P0 ticket from a platinum or gold customer must be assigned to a senior engineer within 30 minutes of opening. Please store this as a new organizational policy.

MA Memory-aware support analyst  Agent · 2:56 PM

tool: platform.context_engine.remember ran

I've stored this as a new organizational policy in memory:

P0 Escalation Policy — Any support ticket with `priority == "P0"` and `customer_tier` in `( "platinum", "gold" )` must be assigned to a senior engineer within 30 minutes of the ticket's opened time.

This is now saved as durable organizational knowledge (memory ID `200e9ba1-16fc-4668-aab7-b214fcdad942`) and will be applied whenever future analysis involves P0 escalation compliance, SLA adherence, or assignment timeliness against the `memory-demo-support-tickets` data.

   12s

Updating a stored definition

Agent Builder demo



TU Test User · 1:07 PM

We have updated our platinum SLO definition. Going forward, a platinum breach is any ticket with customer_tier platinum and resolution_minutes greater than 60. We raised the threshold from 45 to 60 minutes. Please update the stored organizational definition.

MA Memory-aware support analyst  Agent · 1:07 PM

4 tools ran >

Found the stored organizational definition. I'll revise it to reflect the new 60-minute threshold.

2 tools ran >

Done. I've updated the stored organizational definition for the platinum SLO breach:

Previous: `customer_tier == "platinum" AND resolution_minutes > 45`

Now (updated): `customer_tier == "platinum" AND resolution_minutes > 60`
(ignoring priority)

This memory (id `edd930d4-8875-431d-a292-c0d58a3f8a44` , tagged `definition` , `platinum` , `slo` , `support-tickets`) still applies to the `memory-demo-support-tickets` index and notes the threshold change with today's date, so future breach analyses will use the new 60-minute cutoff automatically.

  ⌚ 28s

What a memory can look like

Support demo: a reusable business rule, shown as a simplified record

Platinum SLO breach definition

type: memory.session_fact

customer_tier = platinum
resolution_minutes > 45
Regardless of priority

Recorded alongside it

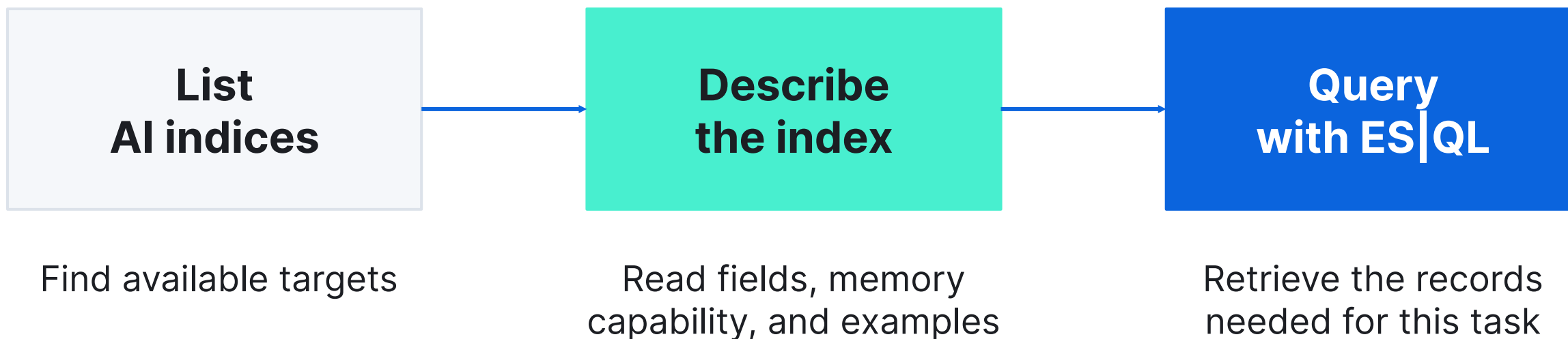
Source conversation
Memory ID and timestamps
Optional expiration

Keep the definition. Query current tickets for the answer.

or

Discover the memory context before querying

Current first-party direction, under development

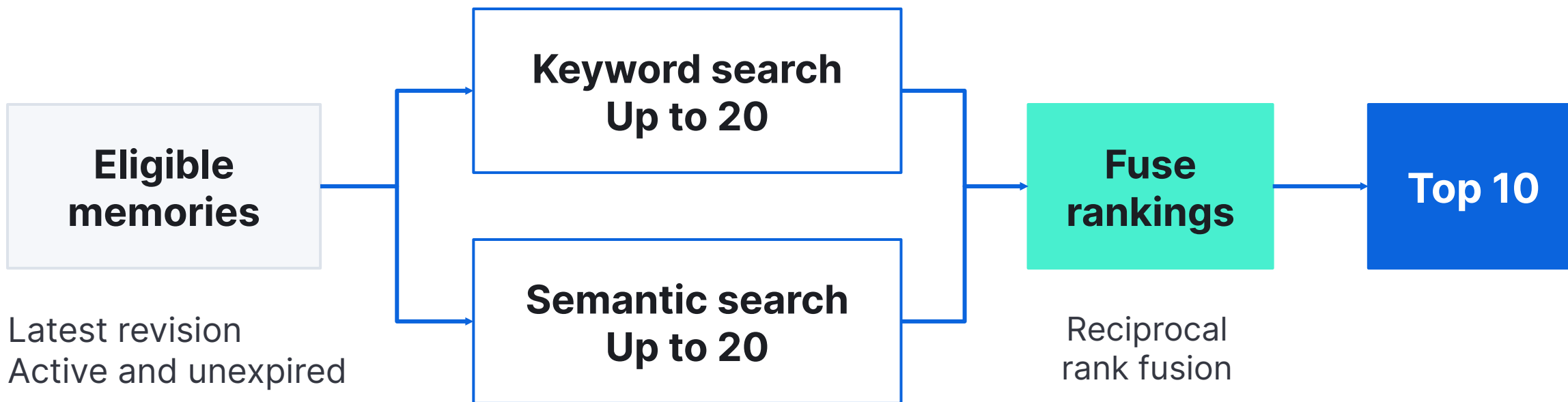


The agent constructs the query. Ordinary retrieval excludes memory by default.

[Read more: Context engineering with Elastic](#)

Hybrid recall combines words and meaning

Built into ES|QL: FORK runs both searches; FUSE combines their rankings.



20 per branch, then up to 10 results: example settings, not fixed API limits.

[Read more: ES|QL FORK / FUSE hybrid search](#)

Top N and a score threshold do different jobs

The current recall example has a result limit, but no explicit minimum-score cutoff.

Maximum memories returned



Return up to 10 ranked memories



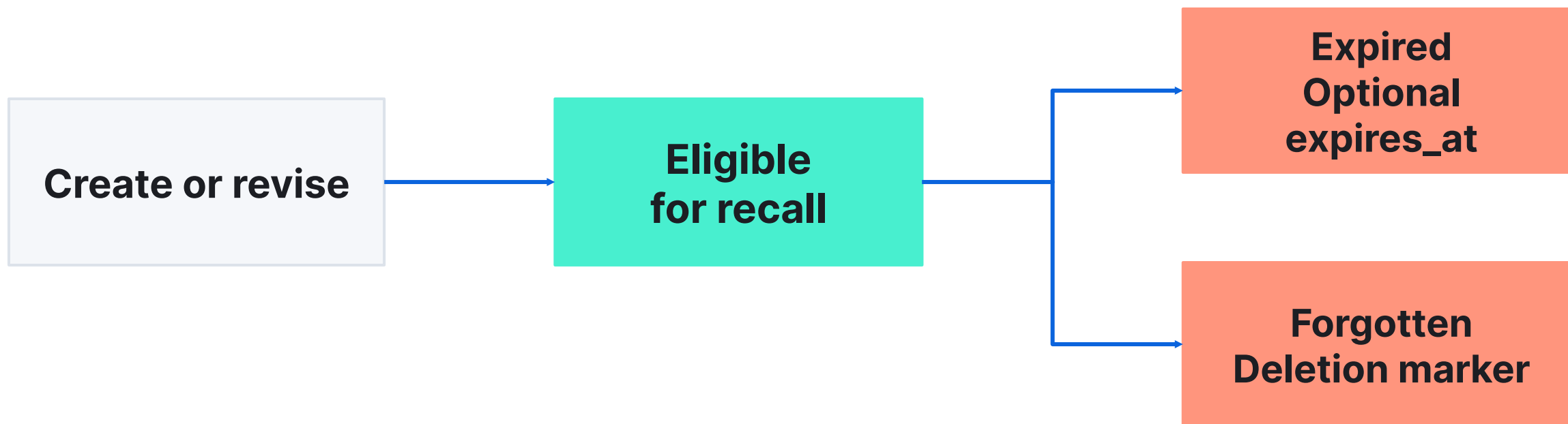
A score threshold is a separate design choice

Best available does not establish useful or correct.

Keeping memory up to date

Memory has a lifecycle

Recall eligibility and physical deletion are different lifecycle concerns.



Recall excludes both

No default TTL in the write tool. Forget requires confirmation and retains a tombstone.

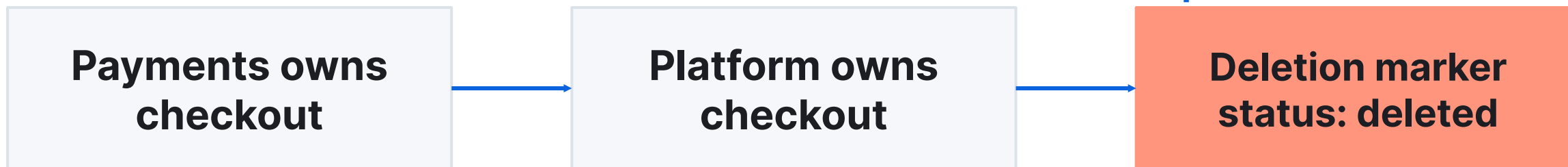
A forget request changes the memory's latest state

Data-stream example: time moves left → right; all records share one memory ID

1 · Oldest: created

2 · Revised

3 · Newest: forget
requested



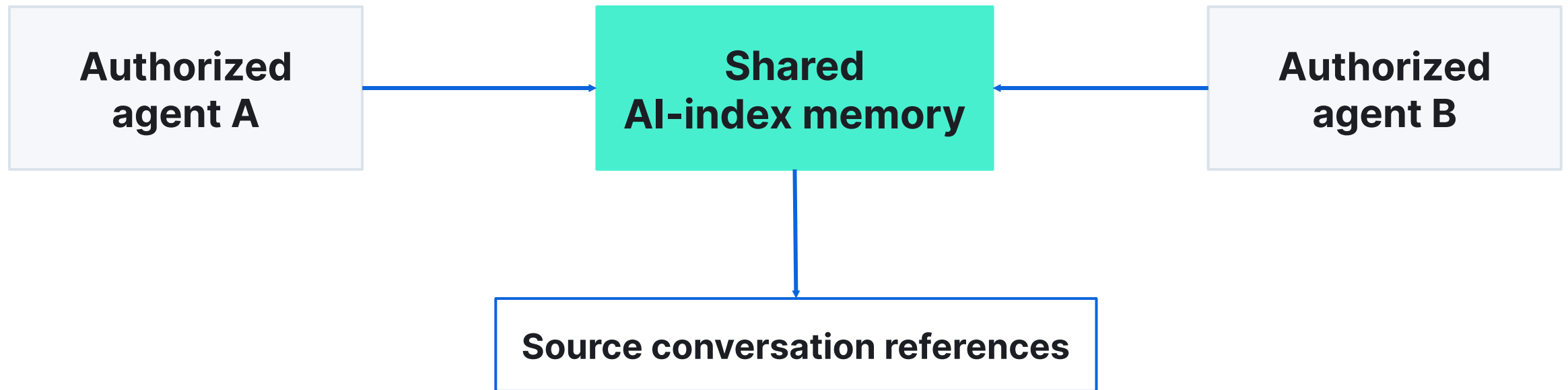
The newest record says: “Do not recall this memory.”

Recall checks that state first. It returns no version of this memory.

Older records stay in the history. Deleting only the first record would leave later versions active.

Sharing boundaries and provenance

Initial scope: shared task knowledge accessible through the AI-index destination



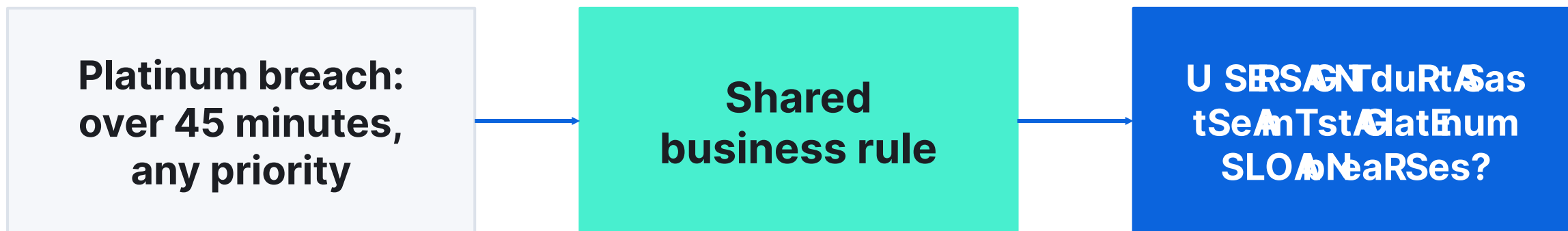
One person's explanation can help the next

Two-user illustration based on the documented cross-conversation demo

User 1 explains

Elastic memory

User 2 asks



Recall the rule + query current tickets → Search: 3 breaches

Both users have access to the shared memory.

Memory saves the user from explaining the rule again

Question: "What is our platinum SLO definition?"

Without the stored definition

**Agent: "What threshold
does your team use?"**

The user explains the rule again.

With shared memory

**Recall the definition
from memory**

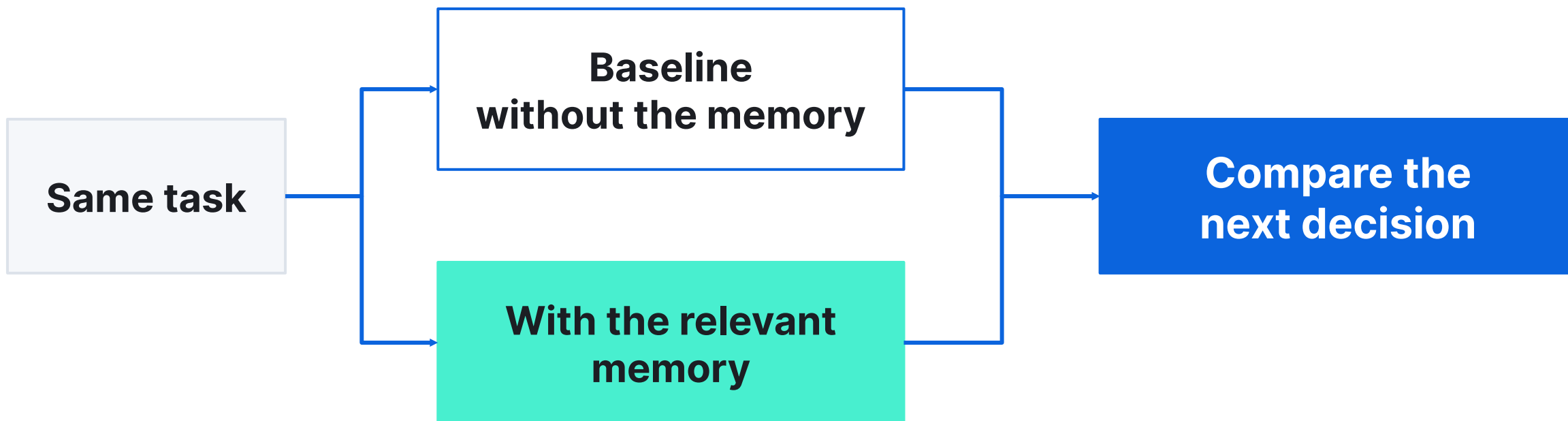
**Answer directly: over 45 minutes,
regardless of priority.**

Saves a repeated explanation. Current ticket counts still require a source-data query.

Evaluating memory and what's next

Evaluate the next decision, not just retrieval

Proposed evaluation design, not measured results



Test corrections, expiry, deletion, irrelevant tasks, and access. Track latency and context size.

The first step and the broader design space

The initial scope gives us concrete behavior to test and learn from.

Initial implementation direction	Broader design questions
Shared task facts and session syntheses	Private preferences, richer sharing, and automatic capture
Agent-written memory and ES QL recall	Consolidation, superseding facts, and contradiction resolution
Optional expiry, logical forgetting, provenance	Adaptive recall and knowledge graphs for entity resolution

Memory is part of Elastic's Context Engine



Capture, recall, lifecycle, and evaluation shape the outcome.

Learn more about Elastic memory

Sign up to learn more.



[https://forms.gle/
ZQhmJ3PW68i6zyvr5](https://forms.gle/ZQhmJ3PW68i6zyvr5)